The tests themselves will also look different from their equivalents in a standard Rails test. For one thing, they aren't called tests but rather *examples*. Also, it's customary for each example to make only one assertion. The existing test of adding different products makes two assertions, so in the spec, that means two examples.

Assertions look different in RSpec as well:

```
expect(actual_value).to eq(expected_value)
```

Applying this to the two assertions around adding distinct items, we have two examples (we'll show you where this code goes in a moment):

```
it "has two line items" do
  expect(cart.line_items.size).to eq(2)
end
it "has a total price of the two items' price" do
  expect(cart.total_price).to eq(book_one.price + book_two.price)
end
```

These assertions won't succeed unless items are added to the cart first. That code *could* go inside each example, but RSpec allows you to extract duplicate setup code into a block using `before`:

```
before do
  cart.add_product(book_one).save!
  cart.add_product(book_two).save!
end
it "has two line items" do
  expect(cart.line_items.size).to eq(2)
end
it "has a total price of the two items' price" do
  expect(cart.total_price).to eq(book_one.price + book_two.price)
end
```

This setup is only relevant to some of the tests of the `add_product` method, specifically the tests around adding different items. To test adding the same item twice, you'll need different setups. To make this happen, wrap the above code in a block using `context`. `context` takes a string that describes the